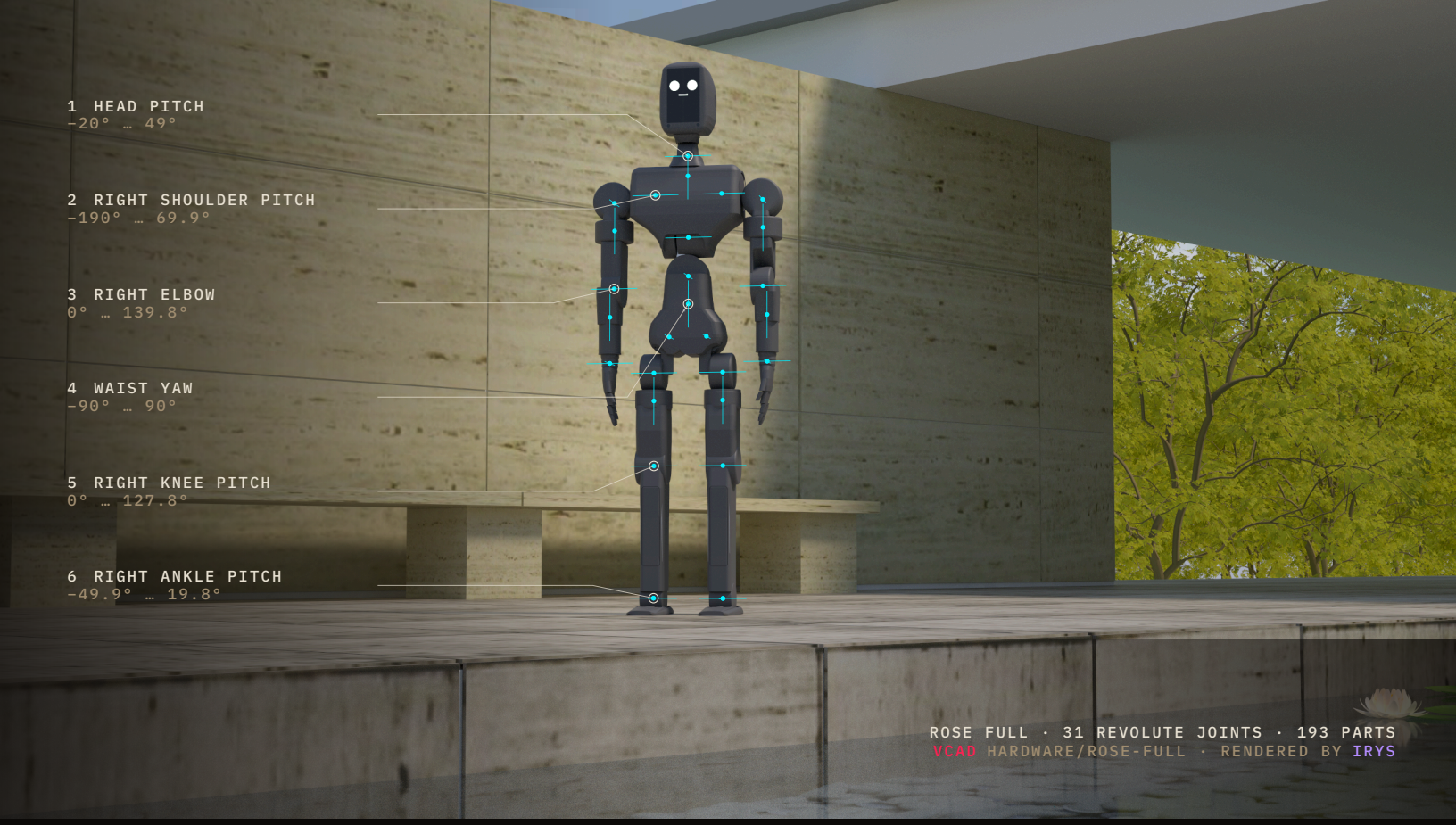



MUNICIPAL ROBOTICS · RESEARCH — NOVEMBER 2026 · DRAFT

# irys

A spectral differentiable renderer with exact replay

CAM PEDERSEN · MUNICIPAL ROBOTICS

Cover: the scene is “Barcelona Pavillion” by eMirage, CC-BY, from blender.org’s demo files.

## Abstract

A renderer turns a scene into an image. A differentiable renderer also says how the image would change if the scene did, and that answer is only as good as the renderer's ability to retrace its own steps. We describe irys, an open-source spectral path tracer in Rust, built for the scenes we design (instanced CAD meshes, spectral metals, robots in rooms) and built to retrace exactly.

Its differentiable core is path replay backpropagation on a GPU wavefront. Materials are written once as graphs in the tang tensor compiler and lowered both to GPU kernels and to an f64 CPU reference. An integer random-number generator, fixed-order reductions and a ray-only BVH walk make the replay reproduce the forward pass bit for bit: 65536 of 65536 lanes, with gradients identical across runs. GPU f32 gradients agree with the f64 reference to $3.5 \times 10^{-4}$. On top of the material gradient sit pose gradients through instanced CAD meshes (primary edges, an interior term with attached frames, and secondary edges that move shadows and reflections) and a per-vertex image gradient that the vcad BRep kernel pulls back to the dimensions of a part.

Every layer is checked against a reference: the GPU's f32 gradients against the f64 tier, the edge terms against central differences with common random numbers, and the shadows a moving part casts against the same differences with and without the secondary term. Through vcad's BRep kernel a per-vertex image gradient becomes the derivative with respect to a part's dimension, matching vcad's forward mode to $1.1 \times 10^{-15}$. We state the limits alongside: replay is bit-exact per device and driver, splats and diffuse surfaces are not yet differentiated, and only the environment emits light. Inverse problems built on irys, starting with editable CAD from photographs, are the subject of a companion paper.

# 1 Introduction

A renderer is judged first by its images. A differentiable renderer is judged also by its gradients: how each pixel would change if a material, a pose or a dimension changed. Those gradients feed optimisers, so an error in them does not show as a wrong pixel; it shows as an optimiser that wanders, and nobody can tell whether the estimator is biased, the device rounded, or the sampler was unlucky.

Path replay backpropagation[1] made physically based differentiable rendering practical by replacing the stored path tape with a second, replayed pass. That only works if the replay regenerates exactly the paths the forward pass drew; otherwise the gradient belongs to a different estimator from the one rendered. Systems built on general array compilers[2] meet this contract inside one process and one backend. We wanted more: a GPU tier and an f64 CPU reference that run the same material source and agree to rounding, with each tier bit-identical across runs. Then a failing gradient test is a bug, never noise.

This paper describes irys, the renderer behind vcad and our robotics tools, and how it keeps that contract. Its contributions are:

- **One BSDF source, two lowerings, two tiers.** Materials are tang tensor graphs. tang lowers each one to an evaluation program and an adjoint program, emitting WGSL kernels for the GPU and running the same graph in its f64 interpreter for the CPU reference. The path logic (ray generation, traversal, bounce, accumulation) is hand-written (Section 3).
- **PRB on a deterministic wavefront with hero wavelengths and NEE.** The estimator is Vicini et al.'s with next-event estimation, as in Mitsuba 3; what is new is a wavefront whose replay is bit-identical, with four wavelengths per path and the light-sampled lanes folded into the same graph batch (Section 4).
- **Bit-exact replay by construction.** An integer RNG that is exact on both tiers, fixed-order reductions with no float atomics, and a BVH walk that depends only on the ray (Section 5).
- **Pose gradients through instanced CAD meshes.** Primary edge sampling with crease ownership and visibility-aware importance, an interior term by forward-mode duals with attached frames, and secondary edge sampling for moving shadows and reflections (Section 6).
- **Validation at every layer.** Gradients against central differences with common random numbers, and GPU against CPU f64, on boxes, on a shaded robot arm read from its CAD source, and on the shadows it casts (Section 7).
- **A bridge to CAD parameters.** A per-vertex image gradient that the vcad BRep kernel pulls back to the dimensions of the part, shared as a benchmark with the companion paper[3] (Section 7.4).

## 1.1 Scope and limits

- **Determinism is per device and driver.** The GPU tier runs in f32, so its results agree with the f64 reference to rounding, not bitwise; it is bit-identical against itself on one device and driver.

- **Materials.** The differentiable material is a GGX conductor with a three-band spectral Fresnel. Diffuse surfaces and the environment are not differentiated. Gaussian splats are rendered but not differentiated.
- **Lights.** The environment is the only emitter (a sky and a von Mises–Fisher sun lobe).
- **Pose parameters are rigid.** Joint angles and the camera move instances rigidly. Non-rigid shape change enters only through the BRep bridge of Section 7.4.
- **Hardware.** All timings come from one Apple M-series machine through Metal; the exact chip was not recorded in the source logs, and timings in this draft are not yet regenerated on a single pinned machine (Section 9).

## 2 Background and related work

**Differentiable path tracing.** Li et al. introduced edge sampling to account for visibility discontinuities in Monte Carlo rendering[4]. Radiative backpropagation[5] and path replay backpropagation[1] reduced the memory of the adjoint pass to a constant per path; Dr.Jit provides a tracing compiler for such methods[2]. Visibility can also be handled by reparameterisation[6], by warped-area sampling[7], or by projective sampling[8]; we compared warped estimators for the secondary term and chose edge sampling (Section 6).

**Spectral rendering.** Hero wavelength sampling[9] traces several wavelengths along one path, the hero choosing directions. We use it unchanged and make each wavelength its own PRB lane.

**Compilers for rendering.** SLANG.D adds automatic differentiation to a shading language[10] and is the closest analogue to writing BSDFs as tang graphs. Laine et al. argue for splitting a path tracer into wavefront kernels[11], the structure we use.

**Differentiable rendering systems.** Mitsuba 3[12] implements path replay with next-event estimation and multiple importance sampling, projective sampling and reparameterisation on Dr.Jit, and is the closest system to irys's differentiable core. irys differs in what it guarantees rather than in the estimator: a replay that is bit-identical to the forward pass by construction, an f64 reference that runs the same material source, spectral transport with hero wavelengths, and visibility terms adapted to CAD tessellations (coplanar slivers, crease ownership, attached frames).

**Inverse CAD and measurement from images.** Recovering an editable CAD description from observations is usually posed as program synthesis over meshes, point clouds or drawings[13–15], and close-range photogrammetry measures manufactured parts by triangulating targets and features[16]. irys supplies what both lack for photographs: exact gradients of a rendered image with respect to a part's pose, finish and dimensions. CAD parameters upstream of the mesh are differentiated by our companion paper's BRep kernel[3], joined to irys in Section 7.4.

## 3 Materials as compiled graphs

The differentiable material has parameters $\theta = (\alpha, F_0^{450}, F_0^{550}, F_0^{650})$: GGX roughness with height-correlated Smith masking, and a Schlick Fresnel whose $F_0$ is interpolated between three spectral hats. It is written once as a tang `TensorGraph` whose inputs are per-lane columns (the cosines at a vertex and the hero wavelength's hat weights) and whose $\theta$ is broadcast. tang lowers it two ways. The **eval** program returns $f$ and its pdf. The **adjoint** program is the backward pass of $\sum_j \delta f_j \cdot f_j$, so the batch sum over lanes is the reduction to $\delta\theta$. Each program is emitted as WGSL, one kernel per materialised comprehension, or as compiled C in f32 and an interpreter in f64 for the CPU. The interpreter matches a hand-written f64 GGX to $10^{-12}$, and the compiled C matches the interpreter to $9.4 \times 10^{-7}$ in the worst relative error.

The path logic is hand-written, in WGSL for the GPU and in Rust for the CPU reference: ray generation, BVH traversal, bounce, the throughput update, the final splat and accumulation, and the device-side loss scoring. tang's comprehensions are straight-line bodies over fixed shapes, so data-dependent path loops and a traversal stack cannot be expressed in them today. Sampling the GGX visible normals is detached, since it needs no derivative under PRB, and the divisor pdf is the sampler's, which we test against the graph's pdf to $10^{-9}$.

## 4 Path replay on a wavefront

**Estimator.** For a path with vertices $j$ and radiance $L$, BSDF-only PRB accumulates $\delta L(L - L_j)\partial_\theta \log f_j$ at each vertex, where $L_j$ is the radiance collected before vertex $j$. With next-event estimation and multiple importance sampling the gradient becomes

$$\partial L/\partial\theta = \sum_j \left[\partial_\theta \log f_j(\omega_{B,j})\left(L - L_j^+\right) + \partial_\theta \log f_j(\omega_{L,j}) C_j\right]$$

where $\omega_B$ is the BSDF-sampled direction, $\omega_L$ the light-sampled one, $C_j$ the NEE contribution at $j$ and $L_j^+$ the radiance up to and including it. The MIS weights are functions of detached pdfs and therefore constants under $\partial_\theta$. In the wavefront the light-sampled lanes are the second half of one graph batch, so a single adjoint dispatch serves both strategies. A nonlinear loss needs the primal image before the adjoint, so the loss is scored between the forward and replay passes; seeding the adjoint from the same pass instead biased roughness from 0.6 to 1.0 in our first spike.

**Hero wavelengths.** Each path carries four stratified wavelengths, the hero and its rotations by $k/4$ of the visible range[9]. Only the hero draws directions. The wavefront's lanes are (path, wavelength) pairs, four times as many as paths, and each lane is its own PRB path. At equal paths, four wavelengths cut per-pixel variance 7.4× in total and 28.7× in chroma.

**Memory.** PRB stores nothing per vertex. With four wavelengths a path costs 63.1 f64 words at depth 2 and at depth 8 alike. Memory scales with paths, not pixels. Frames larger than one dispatch are traced in chunks, and a $1024^2$ image at 4 spp, 16777216 lanes, replays bit-identically in nine chunks.

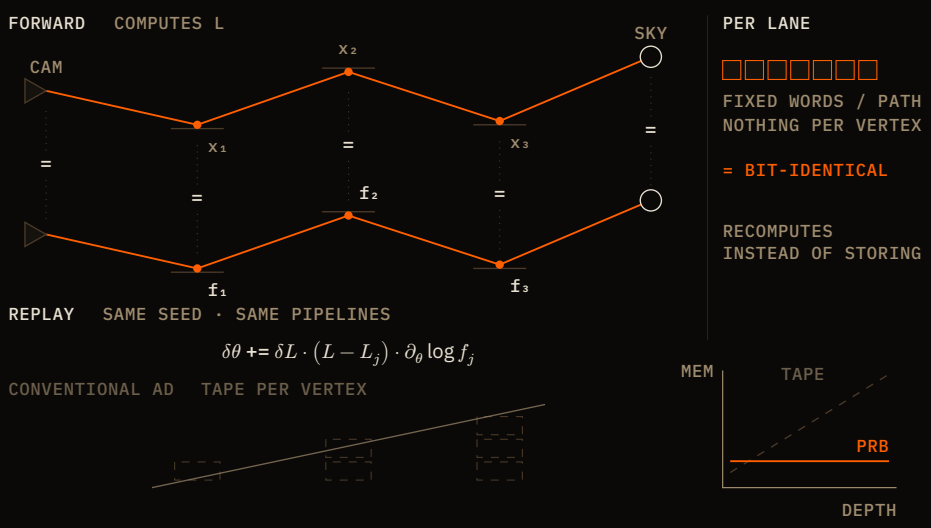



**FIG 1 Path replay, drawn.** The forward pass computes a path's radiance and keeps a fixed handful of words per path, nothing per vertex. The replay regenerates the same path from the same seed through the same pipelines, bit for bit, and accumulates $\delta\theta$ at each vertex. Memory stays constant in depth, where an AD tape would grow with it.

## 5 Determinism

PRB's replay must redraw the forward pass's paths. irys makes this hold in four ways.

- **Same pipelines.** Forward and replay dispatch the same compute pipelines over the same bind groups; they differ only in a uniform that enables the store of $\delta f$.
- **An integer RNG on both tiers.** SplitMix64 is ported onto pairs of 32-bit words built from 16-bit limbs, so the GPU, which lacks 64-bit integers, produces exactly the CPU's stream. A uniform keeps the top 24 of 53 bits, so both tiers walk the same paths up to f32 rounding of the geometry.
- **No float atomics.** The reduction over lanes is tang's monoid reduction, split 256 ways with one workgroup per parameter and a fixed combining tree. Splat accumulation runs in bounce order, loss scoring uses a fixed stride and tree, chunks are summed in order, and the device prefix sum used by edge sampling has a fixed order.
- **A walk that depends only on the ray.** BVH traversal uses a fixed-size stack, visits the near child first by slab entry with a deterministic tie-break, and culls by the closest hit so far. The walk, and therefore the floating-point sequence, is a function of the ray alone, not of thread scheduling.

The result is that the replay reproduces the forward radiance bit for bit on every lane, 65536 of 65536 at depths 2 and 8, with NEE, and through the two-level instanced scenes and a garden scene of meshes and Gaussian splats; two runs give bitwise-identical loss and $\delta\theta$. The guarantee holds per device and driver. A different GPU or compiler may round differently, and then agrees with the CPU reference only to f32 precision.

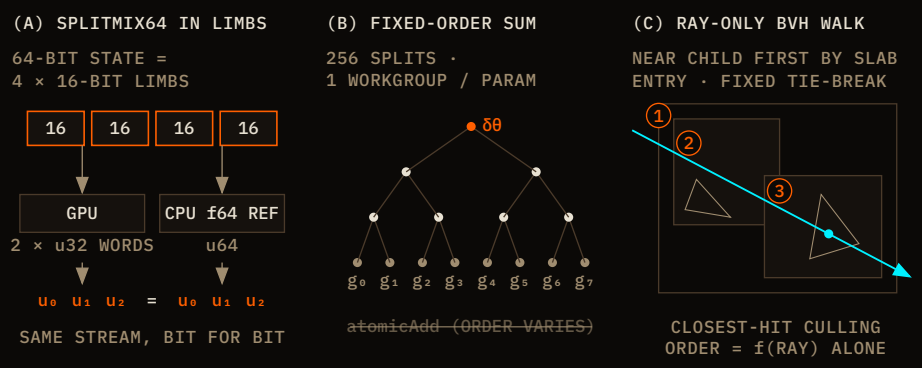
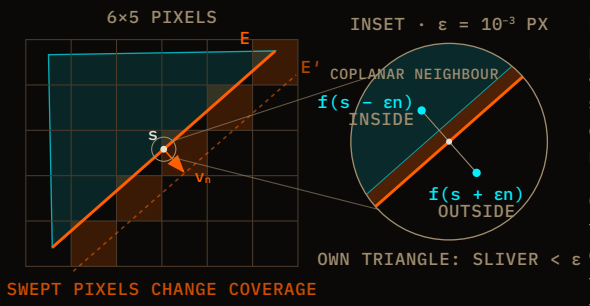



**FIG 2 Three choices make the replay bit-exact.** (a) SplitMix64 on 16-bit limbs gives the GPU, which has no 64-bit integers, the CPU's exact stream. (b) Lane gradients are summed by a fixed tree, never by float atomics. (c) BVH traversal visits children in an order that depends only on the ray.

## 6 Pose gradients through instanced CAD

**Scenes.** A scene is a two-level BVH: one bottom-level tree per CAD part or group, and a top-level tree over instances, each with an object-to-world transform. irys maps rays into an instance's space without renormalising them, so hit distances compare across instances. A pose $\varphi$, such as joint angles, is a map from parameters to the transforms of the moving instances. The full robot in this paper has 2.4 million triangles, tessellated by vcad from its CAD source (a text format of parametric solids).

**Primary edges.** Moving an instance moves the curves across which the radiance arriving at the camera plane jumps: silhouettes, boundaries and creases between faces that are not coplanar. By Reynolds transport, a pixel's derivative includes




Moving E to E′ changes which pixels are covered. The change is a jump, invisible to derivatives of shading at fixed rays; only rays either side of a sampled point on E measure it.

CAD tessellations are full of slivers thinner than ε. The inner ray counts as inside on the edge's own triangle or a coplanar neighbour of the same face.

$$\frac{\partial I_p}{\partial \varphi} \supseteq \int_E (f(s-\varepsilon n) - f(s+\varepsilon n))\, v_n \, \mathrm{d}\ell$$

**FIG 3 Primary edge sampling.** Moving an edge under $\varphi$ changes which pixels it covers, a jump that no derivative of shading at fixed rays can see. Paired rays traced either side of a sampled point measure it. On CAD tessellations the edge's own triangle is often a sliver thinner than $\varepsilon$, so the inner ray counts as inside on any coplanar triangle of the same face. The inset is not to scale.

$$\partial I_p/\partial\varphi \supseteq \int_{E\cap p} (f(s-\varepsilon n) - f(s+\varepsilon n)) v_n(s)\, \mathrm{d}\ell(s)$$

over the projected edges $E$, where $n$ is the edge's outward normal and $v_n$ the normal speed of the material point under $\varphi$[4]. We sample edges with density proportional to screen length times an importance (1 for silhouettes, a dihedral weight for creases,

a small weight for back-facing edges), trace the two sides $10^{-3}$ pixels apart with common random numbers, and require the inner ray to hit the edge's own triangle or a coplanar neighbour. CAD tessellations are full of slivers thinner than $\varepsilon$, and without that coplanar rule the estimator loses most silhouette samples. A crease between two front faces is owned by the face that meets the eye more squarely, so its jump is counted exactly once. A low-resolution buffer of which instance each pixel sees lets the sampler demote edges it shows hidden, which cut variance 1.9× at equal samples. Sampling runs on the device, with a fixed-order prefix sum; it took the edge term on the arm from 105 ms to 7.4 ms per iteration.

**Interior term and attached frames.** Away from discontinuities the estimator is smooth in $\varphi$ at fixed random numbers, and we differentiate it by forward-mode dual numbers through a re-trace of the frame: ray–triangle intersection with the corners as duals, the shading frame, GGX sampling and evaluation, NEE and the MIS weights. The renderer builds a vertex's tangent frame from its normal with a construction that flips sign when the normal crosses a plane, a discontinuity in $\varphi$ at fixed random numbers that no edge term sees. We instead differentiate the estimator whose frame on a moving instance is rigidly attached, $R(\varphi)R(\varphi_0)^T\ \mathrm{basis}(n(\varphi_0))$. At $\varphi_0$ it is the rendered estimator exactly, and for every $\varphi$ it is an equally valid estimator of the same image, so its derivative is an unbiased estimate of $\partial I/\partial\varphi$ from the paths the forward pass already drew. The duals reproduce the forward pass's values to $9.3 \times 10^{-15}$.

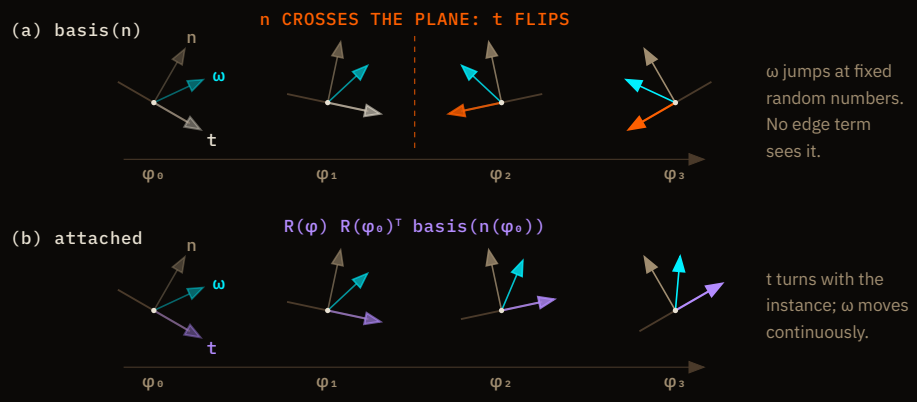



**FIG 4 Attached frames.** Building the tangent frame from the normal flips its sign when the normal crosses a plane, so a direction drawn at fixed random numbers jumps, and no edge term sees it (a). A frame rigidly carried by the instance turns with it (b). Both are valid estimators of the same image; only the attached one is differentiable at fixed random numbers. Drawn in 2D, where the flip is a reversal.

**Secondary edges.** When an instance moves, its shadow and its reflection move too. We sample the edges seen from each path vertex with a per-tree edge BVH, under one-sample multiple importance over three densities: geometric, toward the sun's cone, and toward the mirror lobe. The boundary term includes the velocity of the BSDF-sampled direction, obtained by inverting the sampler at the path's random numbers and redrawing it in duals. Against warped-area estimators on the same configurations, edge sampling had 14–19× lower variance at 1.6× lower cost.

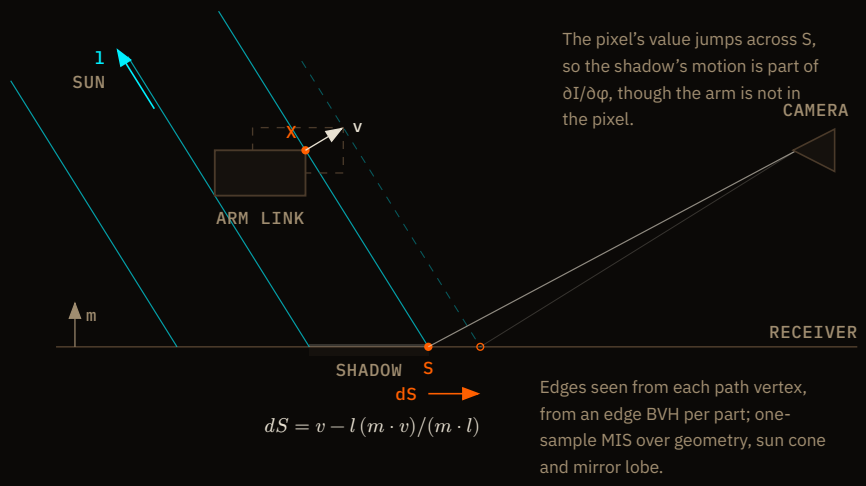



**FIG 5 Moving shadows.** An occluder's light-silhouette edge casts a shadow edge on its receiver; when the occluder moves by $v$, the shadow edge moves by $dS = v - l(m \cdot v)/(m \cdot l)$, and every pixel it crosses changes, whether or not the occluder is in view.

**Camera.** The same machinery differentiates a seven-parameter camera (translation, rotation, field of view), with the eye, axes and field of view as duals and every instance's edges as candidates.

## 7 Results

Unless stated, we check gradients against central differences (CD) of the same estimator with common random numbers; "frozen" means the random numbers are fixed so the CD is noise-free. Monte Carlo comparisons report a z-score, the difference over its standard error. Timings are release builds on the M-series machine above. The number in each table is read from `data/measured.tsv`, whose rows give the source test and commit.

### 7.1 Correctness of the material gradient

**TABLE 1 Material gradient, layer by layer.** Relative errors. The GPU's worst lanes are silhouette rays, where f32 geometry moves a hit across an edge. `data/measured.tsv`.

| CHECK | CONFIGURATION | RESULT |
|---|---|---|
| **CPU f64 PRB vs frozen CD** | BSDF-only, four θ | $1.6 \times 10^{-10}$–$1.9 \times 10^{-9}$ |
| **same, hero wavelengths** | 4 λ | $\le 6.1 \times 10^{-10}$ |
| **same, NEE + MIS** | two instanced boxes | $\le 3.1 \times 10^{-9}$ |
| **GPU f32 vs CPU f64, radiance** | median / p99 per lane | $1.2 \times 10^{-7}$ / $1.5 \times 10^{-4}$ |
| **GPU f32 vs CPU f64, loss** | 32×32×4 | $5.8 \times 10^{-7}$ |
| **GPU f32 vs CPU f64, δθ** | worst parameter | $3.5 \times 10^{-4}$ |
| **same with NEE** | radiance p99 / worst δθ | $4.1 \times 10^{-6}$ / $3.6 \times 10^{-6}$ |
| **Replay vs forward** | 64×64×4×4 λ, depth 2 and 8 | 65536/65536 bit-identical |

### 7.2 Variance and throughput

At equal samples per pixel, NEE with MIS lowers per-pixel variance 4.5× against BSDF sampling alone on a scene of two instanced boxes, with image means agreeing within noise. A gradient of the 96×96, 8 spp frame at depth 4 (73728 paths, four wavelengths each) costs 7.5 ms on the GPU without NEE and 14.9 ms with it; the forward pass alone takes 0.87 ms, about 85 million paths per second. The same gradient through the compiled-C CPU tier costs 116–129 ms, so the GPU tier is about 15× faster end to end. Most of the gradient's time is the replay, because the adjoint's four-parameter reduction is not yet fused and NEE doubles the batch.

### 7.3 Pose gradients

On a box against the sky, where the edge term is the whole derivative, edge sampling matches CD within 3.3, 0.8 and 1.3 standard errors on its three pose parameters, and the device's f32 edge term matches the CPU's f64 to $1.8 \times 10^{-6}$. On the shaded arm over a floor, the edge and interior terms alone miss CD by z = 3.8, −3.9, 3.4: the arm's shadow on the floor carries about 15% of the gradient. With secondary edges the same check gives z = 0.77, −1.07, 0.06.

### 7.4 From pixels to CAD parameters

The edge sampler's velocity is a projection of the material point's motion, so replacing the pose's velocity with a unit translation per axis yields $\partial \mathcal{L} / \partial x_i$ for every mesh vertex. vcad's differentiable seam[3] takes exactly that cotangent on the nodes of a frozen tessellation and returns the derivative with respect to a CAD parameter in one reverse pass. The shared benchmark uses rose's left shoulder-pitch output plate, a 3 mm rounded plate with a 49 mm bearing bore and twelve bolt holes, with $\theta$ the bore diameter. irys renders vcad's frozen tessellation, the mesh whose connectivity the kernel holds fixed while it differentiates node positions (4942 nodes, 9932 triangles) as it is, against the sky at 160×128, under a fixed smooth image loss.

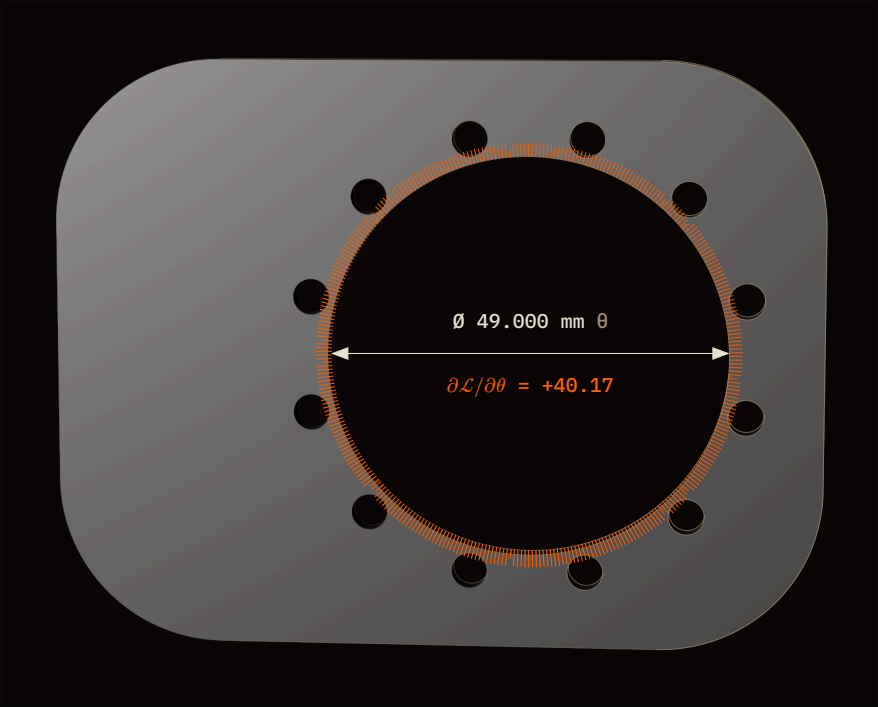



**FIG 6** **Where the bore's gradient comes from.** irys's render of the benchmark plate with vcad's frozen mesh fading in from left to right. Each tick sits on a node that moves when the bore grows, points the way that node moves on screen, and is as long as its share of $\partial\mathcal{L}/\partial\theta$. Only the bore's nodes move with $\theta$, so the derivative picks out exactly that feature. Every mark is data from the run in Table 2.

**TABLE 2** **The shared rose-arm benchmark** at irys `103e3da`, vcad `d10f279`, rose `832192f`. The pullback's error is the Monte Carlo standard error over 8 edge-sampling seeds of 100 000 samples. The render FD moves the plan's nodes along vcad's FD velocities; it varies by about 0.5% between $h = 0.02$ and 0.1 mm, so the pullback agrees with it within the FD's own error. The same numbers appear in the companion paper.

| PATH TO DL/DΘ | VALUE | VS PULLBACK |
|---|---|---|
| vcad pullback of irys's $\partial\mathcal{L}/\partial x$ | 40.16864 ± 0.0226 | – |
| vcad forward seam, $\sum \partial\mathcal{L}/\partial x \cdot \dot{x}$ | 40.16864 | $1.1 \times 10^{-15}$ |
| vcad finite-difference node velocities | 40.17028 | $4.3 \times 10^{-5}$ |
| End-to-end render FD ($h = 0.1$ mm) | 40.0144 | $3.9 \times 10^{-3}$ |

A finite difference that rebuilds the part is not a usable reference here. Rebuilding at $\theta \pm h$ loses node correspondence for $h \geq 2 \times 10^{-3}$ mm and changes topology at 0.05 mm, where the bore rim's segment count, chosen from the circle's size, drops by one; below that the render difference is quadrature noise. The pullback has no step size. It is exact for the frozen mesh, and irys's sampling error is the only noise in the result.

## 8 Discussion

**Why insist on bit-exact replay.** With an exact replay, a gradient that disagrees with its reference is a bug in the estimator, not an artefact of the replay. We could therefore set every gate in this paper tight and debug failures by bisection. That is how we

found the crease-ownership bug in our first edge sampler. It inflated one parameter's edge term by a factor of five on a shaded arm while the mask-only test passed.

**The f64 reference.** Running the same tang graph in f64 on the CPU separates three error sources that are otherwise entangled: estimator bias (CPU against frozen CD), device rounding (GPU against CPU), and Monte Carlo noise (live sampler against CD over seeds).

**Future work: from photographs to CAD programs.** This paper validates gradients; it does not use them to recover anything. The inverse problems are where they pay off, and the hardest is the model itself: given a few photographs of a part, find the editable CAD program, its features and their dimensions, with error bars. irys's gradients make the dimensions tractable inside such a search, and a printed tag fixes the camera and the scale. What remains open is topology: which features the part has. We take that up in a companion paper[17], with a held-out evaluation and a continuous relaxation over a program's optional features.

**Remaining gaps.** The secondary term runs on the CPU in f64 and costs about a quarter of an iteration; its estimate is heavy-tailed, and the rose check passes at 64 seeds but not at 256. The device edge sampler is biased low on the robot by a few percent for a cause we have not found, so the robot checks use host samples. The adjoint's reduction is not fused. Splats are not differentiated, and only the environment emits light.

## 9 Reproducibility

irys is open source under Apache-2.0. Each row of `data/measured.tsv` names the test or example in `crates/irys-diff` that measures it and the commit that introduced it; the robot-arm checks and the BRep benchmark need the rose CAD checkout. This draft's rows are collected from irys's design log at the commits listed. They were measured on different days, and every timing quoted is a release build. Before submission they will be regenerated in one pass on one recorded machine by a script in `paper/irys/repro`, as for the companion paper.

## 10 Conclusion

A differentiable renderer is only as trustworthy as its replay. irys makes the replay exact by construction: an integer random-number generator, fixed-order reductions and a ray-only traversal reproduce the forward pass bit for bit, materials compile from one graph to a GPU tier and an f64 reference, and pose and shape gradients run through the instanced CAD a kernel produces, down to the dimensions in the model. Every gradient in this paper is checked against an independent reference. What those gradients can recover, starting with editable CAD from photographs, is the companion paper's subject. irys is open source.

## References


1. Vicini, D., Speierer, S. & Jakob, W. Path Replay Backpropagation: Differentiating Light Paths using Constant Memory and Linear Time. *ACM Transactions on Graphics* **40,** (2021).
2. Jakob, W., Speierer, S., Roussel, N. & Vicini, D. Dr.Jit: A Just-In-Time Compiler for Differentiable Rendering. *ACM Transactions on Graphics* **41,** (2022).
3. Pedersen, C. Exact Gradients Through a BRep Kernel. (2026).
4. Li, T.-M., Aittala, M., Durand, F. & Lehtinen, J. Differentiable Monte Carlo Ray Tracing through Edge Sampling. *ACM Transactions on Graphics* **37,** (2018).
5. Nimier-David, M., Speierer, S., Ruiz, B. & Jakob, W. Radiative Backpropagation: An Adjoint Method for Lightning-Fast Differentiable Rendering. *ACM Transactions on Graphics* **39,** (2020).
6. Loubet, G., Holzschuch, N. & Jakob, W. Reparameterizing Discontinuous Integrands for Differentiable Rendering. *ACM Transactions on Graphics* **38,** (2019).
7. Bangaru, S. P., Li, T.-M. & Durand, F. Unbiased Warped-Area Sampling for Differentiable Rendering. *ACM Transactions on Graphics* **39,** (2020).
8. Zhang, Z., Roussel, N. & Jakob, W. Projective Sampling for Differentiable Rendering of Geometry. *ACM Transactions on Graphics* **42,** (2023).
9. Wilkie, A., Nawaz, S., Droske, M., Weidlich, A. & Hanika, J. Hero Wavelength Spectral Sampling. *Computer Graphics Forum (Proc. Eurographics Symposium on Rendering)* **33,** (2014).
10. Bangaru, S. P. *et al.* SLANG.D: Fast, Modular and Differentiable Shader Programming. *ACM Transactions on Graphics* **42,** (2023).
11. Laine, S., Karras, T. & Aila, T. Megakernels Considered Harmful: Wavefront Path Tracing on GPUs. in *High-Performance Graphics* (2013). doi:10.1145/2492045.2492060.
12. Jakob, W. *et al.* Mitsuba 3 renderer. https://mitsuba-renderer.org/ (2022).
13. Sharma, G., Goyal, R., Liu, D., Kalogerakis, E. & Maji, S. CSGNet: Neural Shape Parser for Constructive Solid Geometry. in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* 5515–5523 (2018).
14. Du, T. *et al.* InverseCSG: Automatic Conversion of 3D Models to CSG Trees. *ACM Transactions on Graphics* **37,** (2018).
15. Ellis, K., Ritchie, D., Solar-Lezama, A. & Tenenbaum, J. B. Learning to Infer Graphics Programs from Hand-Drawn Images. in *Advances in Neural Information Processing Systems (NeurIPS)* vol. 31 (2018).
16. Luhmann, T., Robson, S., Kyle, S. & Boehm, J. *Close-Range Photogrammetry and 3D Imaging*. (De Gruyter, 2019).
17. Pedersen, C. Seeing Is Measuring. (2026).